PYTHON FOR BEGINNERS · SESSION 3

Strings and Operators

Work with text using indexing, slicing, string methods and f-strings, then compare values with operators.

Recap

Ask the class:

- What does `input()` always return?
- What is the output of `int("5") + 5`?
- What is the difference between `/` and `//`?

What is a string?

A string is a **sequence of characters** inside quotes.

strings.py

```
a = 'Python'
b = "Python"
c = """This is
a multi-line
string"""
print(a, b)
print(c)
```

OUTPUT

```
Python Python
This is
a multi-line
string
```

Quotes

- **Single and double quotes** work the same way.
- Use **triple quotes** for multi-line text.
- You can **mix quotes** to put one kind inside the other.

```
print("It's a good day")
print('He said "Hello"')
```

OUTPUT

```
It's a good day
He said "Hello"
```

Escape characters

```
print("Line 1\nLine 2")
print("Name\tAge")
print("He said \"Hi\"")
print("C:\\Users")
```

OUTPUT

```
Line 1
Line 2
Name    Age
He said "Hi"
C:\Users
```

Escape table

ESCAPE	MEANING
`\n`	new line
`\t`	tab
`\\`	backslash
`\"`	double quote

Ask

How do you print `I'm "happy"`?

Indexing

Every character has a position, called its **index**, and counting starts from **0**.

Index positions

```
 P   y   t   h   o   n
 0   1   2   3   4   5
-6  -5  -4  -3  -2  -1
```

```
s = "Python"
print(s[0])
print(s[3])
print(s[-1])
print(s[-3])
```

OUTPUT

```
P
h
n
h
```

Ask

What is `s[-2]`?

Two errors to know

- `s[6]` gives an **IndexError**, because there is no position 6.
- Strings are **immutable**: `s[0] = "J"` gives a **TypeError**. You cannot change a character in place.

Try it: read the error

```
s = "Python"
print(s[6])
```

ERROR

```
  File "your code", line 2, in <module>
IndexError: string index out of range
```

Try it: read the error

```
s = "Python"
s[0] = "J"
```

ERROR

```
  File "your code", line 2, in <module>
TypeError: 'str' object does not support item assignment
```

Slicing

Syntax: `string[start:stop:step]`. The **stop index is not included**.

```python
s = "Python Programming"
print(s[0:6])
print(s[:6])
print(s[7:])
print(s[:])
print(s[::2])
print(s[::-1])
print(s[-11:])
```

OUTPUT

```
Python
Python
Programming
Python Programming
Pto rgamn
gnimmargorP nohtyP
Programming
```

How slicing works

- Leaving out **start** means from the beginning.
- Leaving out **stop** means up to the end.
- A step of **-1** reverses the string.

Common string functions and methods

`len()` is a **function**. The rest are **methods**, called with a dot.

```python
s = "  Hello Python  "
print(len(s))
```

OUTPUT

```
16
```

Changing the text

METHOD	PURPOSE	EXAMPLE
`upper()`	all capitals	`"abc".upper()` gives `ABC`
`lower()`	all small	`"ABC".lower()` gives `abc`
`title()`	first letter of each word capital	`"hello world".title()`
`capitalize()`	first letter of the string capital	`"hello world".capitalize()`
`strip()`	remove spaces at both ends	`" hi ".strip()`
`lstrip()` / `rstrip()`	remove left or right spaces	
`replace(a, b)`	replace text	`"cat".replace("c", "b")`

Searching and checking

METHOD	PURPOSE	EXAMPLE
`find(x)`	first index, or -1 if not found	`"hello".find("l")`
`count(x)`	number of occurrences	`"banana".count("a")`
`startswith(x)`	True or False	`"python".startswith("py")`
`endswith(x)`	True or False	`"file.py".endswith(".py")`
`isdigit()`	only digits?	`"123".isdigit()`
`isalpha()`	only letters?	`"abc".isalpha()`

Splitting and joining

METHOD	PURPOSE	EXAMPLE
`split()`	break into a list	`"a b c".split()`
`join()`	join a list into a string	`"-".join(["a","b"])`

Try the methods

```
text = "python is easy"
print(text.upper())
print(text.title())
print(text.replace("easy", "powerful"))
print(text.split())
print(text.count("s"))
```

OUTPUT

```
PYTHON IS EASY
Python Is Easy
python is powerful
['python', 'is', 'easy']
2
```

Methods return a new string

Important

Methods **return a new string**. They do not change the original.

```
name = "aman"
name.upper()
print(name)
name = name.upper()
print(name)
```

OUTPUT

```
aman
AMAN
```

String operations

```
a = "Hello"
b = "World"
print(a + b)
print(a + " " + b)
print(a * 3)
print("a" in "apple")
print("z" not in "apple")
```

OUTPUT

```
HelloWorld
Hello World
HelloHelloHello
True
True
```

- `+` is **concatenation**.
- `*` is **repetition**.
- `in` checks whether something is **present**.

Mixing a string and a number

```
age = 25
print("Age: " + str(age))
```

OUTPUT

```
Age: 25
```

f-strings (formatted strings)

The **cleanest way** to mix text and variables.

```
name = "Aman"
age = 25
print(f"My name is {name} and I am {age} years old")
print(f"Next year I will be {age + 1}")
```

OUTPUT

```
My name is Aman and I am 25 years old
Next year I will be 26
```

Formatting numbers

```
price = 49.5678
print(f"Price: {price:.2f}")
print(f"{1000000:,}")
```

```
OUTPUT
Price: 49.57
1,000,000
```

- `.2f` shows **2 decimal places**.
- `,` adds **thousand separators**.

Operators

Arithmetic (quick revision)

```
print(10 + 3, 10 - 3, 10 * 3, 10 / 3, 10 // 3, 10 % 3, 10 ** 3)
```

```
OUTPUT
13 7 30 3.3333333333333335 3 1 1000
```

Comparison operators

The result is always **True or False**.

```
a, b = 10, 20
print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a >= 10)
print(a <= 5)
```

```
OUTPUT
False
True
False
True
True
False
```

Classic beginner mistake

`=` **assigns** and `==` **compares**. Do not mix them up.

Logical operators

```
age = 22
marks = 80
print(age > 18 and marks > 75)
print(age > 25 or marks > 75)
print(not age > 18)
```

OUTPUT

```
True
True
False
```

OPERATOR	MEANING
`and`	True only if both are True
`or`	True if at least one is True
`not`	reverses the result

Truth table

A	B	A AND B	A OR B
T	T	T	T
T	F	F	T
F	T	F	T
F	F	F	F

Comparing strings

```
print("apple" == "apple")
print("Apple" == "apple")
print("a" < "b")
```

OUTPUT

```
True
False
True
```

Comparison is **case-sensitive**.

Membership and identity (brief)

```
print("py" in "python")
x = None
print(x is None)
```

OUTPUT

```
True
True
```

Operator precedence (high to low)

1. `()`
2. `**`
3. `* / // %`
4. `+ -`
5. comparison operators
6. `not`
7. `and`
8. `or`

```
print(5 + 3 > 7 and 10 % 3 == 1)
```

OUTPUT

```
True
```

Practice questions

Strings

1. Take a name and print its length, uppercase, lowercase and reversed form.
2. Take a name and print the first letter, the last letter and the middle character.
3. Take a sentence and print the number of words (use `split()` and `len()`).
4. Take an email address and print the username and domain (use `split("@")`).
5. Count how many times `a` appears in a user-entered word.
6. Take a string with extra spaces and print it cleaned and in title case.

f-strings and operators

7. Print `Name: ... | Age: ... | City: ...` using an f-string.
8. Take two numbers and print whether the first is greater, equal, or smaller, as True/False values.
9. Take age and print whether the person is eligible to vote (`age >= 18`).
10. Take a number and check whether it is even (`n % 2 == 0`).

Find the output

Work out the output first, then press Run to check

```
s = "Programming"
print(s[2:6])
print(s[::-1])
print(s[-3:])
```

Try the first question

Your turn: finish the program, then press Run

```
name = "Aman"
# print the length, uppercase, lowercase and reversed form
print(len(name))
```

Homework and wrap-up

- Take a full name (for example `aman kumar raj`) and print it in **title case**, then print the **initials** (`A.K.R.`).
- Write a program that takes a word and prints the **first 3 and last 3 letters**.
- Write a program that takes marks in 3 subjects, prints the **average with 2 decimals** using an f-string, and prints `True` or `False` for whether the average is **at least 40**.
- Revise the **string methods** table.

Next class

Conditional statements (`if`, `elif`, `else`).